

Chapter 9: MAP (Most Important STL for Interviews)

If I could teach only **one STL container for interviews**, it would be:

```
map
```

Because frequency counting, dictionaries, grouping, counting occurrences, indexing, and many interview questions use Map.

What Problem Does Map Solve?

Suppose:

```
Rahul -> 95  
Amit  -> 88  
Neha  -> 91
```

You need:

```
Key    -> Value
```

Use:

```
map<string,int> mp;
```

Real World Example

```
Roll Number -> Marks  
Word         -> Frequency  
Country      -> Population  
Employee ID  -> Salary
```

Map stores:

```
key -> value
```

Internal Structure

Usually:

```
Red Black Tree
```

Therefore:

```
insert  
erase  
find
```

all are:

```
 $O(\log n)$ 
```

Header

```
#include <map>
```

Declaration

```
map<string, int> mp;
```

Inserting Data

Method 1

```
mp["Rahul"] = 95;  
mp["Amit"] = 88;  
mp["Neha"] = 91;
```

Result:

```
Amit -> 88  
Neha -> 91  
Rahul -> 95
```

Notice:

Automatically sorted by key.

Method 2

Using insert()

```
mp.insert({"Rahul", 95});  
mp.insert({"Amit", 88});
```

Access Values

```
cout << mp["Rahul"];
```

Output:

```
95
```

Important Behavior

```
cout << mp["Unknown"];
```

Output:

```
0
```

Why?

Because map creates key automatically.

After this:

```
Unknown -> 0
```

exists in map.

Interview favorite.

Traversing Map

Range Loop

```
for(auto x:mp)
{
    cout<<x.first<<" "
    <<x.second<<endl;
}
```

Output:

```
Amit 88
Neha 91
Rahul 95
```

Iterator

```
for(auto it=mp.begin();
    it!=mp.end();
    it++)
```

```
{  
    cout<<it->first<<" ";  
    cout<<it->second<<endl;  
}
```

Important:

Use:

```
it->first  
it->second
```

not

```
it.first
```

Size

```
mp.size();
```

Empty

```
mp.empty();
```

Find

Most important function.

find()

```
auto it = mp.find("Rahul");
```

If found:

```
it != mp.end()
```

Example

```
if(mp.find("Rahul")!=mp.end())  
{  
    cout<<"Found";  
}
```

Complexity

```
O(log n)
```

Count

Check existence.

```
mp.count("Rahul");
```

Output:

```
1
```

or

```
0
```

Complexity

```
O(log n)
```

Erase

By Key

```
mp.erase("Rahul");
```

By Iterator

```
auto it=mp.find("Rahul");  
mp.erase(it);
```

Complexity

```
O(log n)
```

Clear

```
mp.clear();
```

Swap

```
mp1.swap(mp2);
```

Lower Bound

Returns first key:

```
>= key
```

Example

```
map<int,string> mp;  
  
mp[10]="A";  
mp[20]="B";  
mp[30]="C";
```

```
auto it=mp.lower_bound(25);
```

Output:

```
30
```

Upper Bound

Returns first key:

```
> key
```

Example

```
auto it=mp.upper_bound(20);
```

Output:

```
30
```

Frequency Counting

The #1 use of Map.

Input:

```
1 2 2 3 1 1
```

Code:

```
map<int,int> freq;  
  
for(int x:arr)  
{  
    freq[x]++;  
}
```

Result:

```
1 -> 3  
2 -> 2  
3 -> 1
```

Why This Works?

Initially:

```
freq[1]
```

doesn't exist.

Map creates:

```
1 -> 0
```

Then:

```
freq[1]++;
```

becomes:

```
1 -> 1
```

Character Frequency

Input:

```
banana
```

Code:

```
map<char,int> freq;  
  
for(char ch:s)  
{  
    freq[ch]++;  
}
```

Output:

```
a -> 3  
b -> 1  
n -> 2
```

Time Complexity Table

Operation	Complexity
insert	$O(\log n)$
erase	$O(\log n)$
find	$O(\log n)$
count	$O(\log n)$
lower_bound	$O(\log n)$
upper_bound	$O(\log n)$

Operation	Complexity
$\square$	$O(\log n)$

Set vs Map

Set	Map
Stores only key	Stores key,value
10	10→50
Unique key	Unique key

Example

Set:

```
set<int> s;
```

Map:

```
map<int,int> mp;
```

Common Interview Uses

Frequency Count

```
map<int,int> freq;
```

Character Count

```
map<char,int>
```

Word Count

```
map<string,int>
```

Index Mapping

```
value -> index
```

Practice Questions

Easy

Q1

Store:

```
Rahul 95  
Amit 88  
Neha 91
```

Print all.

Q2

Find marks of Rahul.

Q3

Check if Amit exists.

Q4

Delete Rahul.

Q5

Count frequency.

Input:

```
1 2 2 3 1 1
```

Output:

```
1 -> 3  
2 -> 2  
3 -> 1
```

Medium

Q6

Character frequency.

Input:

```
banana
```

Q7

Word frequency.

Input:

```
i love cpp i love dsa
```

Output:

```
i -> 2  
love -> 2  
cpp -> 1  
dsa -> 1
```

Q8

Find first non-repeating character.

Q9

Find most frequent element.

Q10

Sort array by frequency.

Interview Level

Q11

Two Sum

LeetCode #1

Q12

Valid Anagram

LeetCode #242

Q13

Majority Element

LeetCode #169

Q14

Top K Frequent Elements

LeetCode #347

Q15

Group Anagrams

LeetCode #49

Common Mistakes

Mistake 1

```
mp["x"]
```

for checking existence.

This inserts key.

Wrong if you only want checking.

Use:

```
mp.find("x")
```

or

```
mp.count("x")
```

Mistake 2

```
it.first
```

Wrong.

Use:

```
it->first  
it->second
```

Mistake 3

Expecting insertion order.

Map is sorted by key.

Revision Sheet

Header

```
#include <map>
```

Declaration

```
map<Key,Value> mp;
```

Examples:

```
map<int,int>  
map<string,int>  
map<char,int>
```

Insert

```
mp[key]=value;  
mp.insert({key,value});
```

Access

```
mp[key]
```

Search

```
mp.find(key);  
mp.count(key);
```

Remove

```
mp.erase(key);
```

Bounds

```
mp.lower_bound(key);  
mp.upper_bound(key);
```

Utilities

```
mp.size();  
mp.empty();  
mp.clear();  
mp.swap();
```

Complexity

insert	$O(\log n)$
erase	$O(\log n)$
find	$O(\log n)$
count	$O(\log n)$
[]	$O(\log n)$

Mental Trigger

When you see:

- Frequency Count
- Character Count
- Word Count
- Key $\rightarrow$ Value
- Dictionary
- Grouping

Think:

```
MAP
```

Assignment

Solve:

1. Frequency Count.
 2. Character Frequency.
 3. Word Frequency.
 4. First Non-Repeating Character.
 5. Most Frequent Element.
 6. Two Sum.
 7. Valid Anagram.
 8. Majority Element.
-